Decision Models as Universal Tool Selectors

Aziz Muminov Alexandra Ilina

November 2026

Abstract

Agents increasingly work through many tools: a coding agent connected to a company's MCP servers can face hundreds or thousands of tool definitions, too many to load into every request. Shipped harnesses therefore let the agent search for tools, mostly with BM25, as in Codex, Anthropic's tool search and Hermes Agent. Lexical search is imprecise, and every extra search is a full agent turn. We propose decision models, which return a probability per option in one forward pass, as general-purpose tool selectors. Our selector, built on Microsoft Decision 1, chooses the service, then the tools, with a "none" option and usage-based names, in about three decision-model calls per search. Inside Hermes Agent, on 37 coding tasks over the real tool definitions of 30 MCP servers, pre-planned paired runs made at the same time used 12.6% fewer tokens than BM25 with DeepSeek V4.1 Flash over 555 pairs, $p = 0.0002$, and 9.7% fewer with GPT-6.1 Sol over 185 pairs, $p = 0.004$, with 36–60% fewer searches and equal pass rates. Offline, the selector put the right tool first for 92% of hand-written queries against 49% for BM25. On the ToolRet benchmark, 7,961 queries over 44,453 tools, the same decision model re-ranking candidates from BM25 and embeddings scored 53.6 NDCG@10, above every published retriever and re-ranker, including models trained for tool retrieval.

1 Introduction

Through the Model Context Protocol, MCP [18], an agent can read CI logs, error trackers, tickets, chat, dashboards, clusters and databases. Each server exposes tens to hundreds of tools and agents connect to many servers, so the definitions alone can fill the context: five common servers already take about 55k tokens before any work [1]. Harnesses therefore keep definitions out of the prompt and let the agent search for tools on demand, mostly with BM25 [1, 20, 22]. BM25 needs no model, but it is imprecise, so agents search repeatedly, and each search is a full LLM turn that re-reads the whole context.

Decision models, or DMs, read a state and a question with named options and return a probability per option in one forward pass, without generating text. Choosing among named options is what tool selection asks for. Our thesis is that a DM can serve as a drop-in, harness-agnostic tool selector: no training, no change to the agent, only the query and the catalog. We test it with Microsoft Decision 1 [16] and make three contributions:

- a DM tool selector, selector F, that chooses services first, with a "none" option, and then tools (§3);
- a test of selector F inside a shipped agent, Hermes Agent, against its built-in BM25, on graded coding tasks over the real tool definitions of 30 MCP servers, with a protocol for measuring token differences between agent configurations (§4–§6.1);
- an offline study of the selector's design and a comparison with published retrievers on the ToolRet benchmark (§6.3–§6.5).

2 Background

How harnesses select tools. Cursor, Cline and opencode load every definition into every request. Codex [22], Anthropic's tool search [1] and Hermes Agent [20] search with Okapi BM25 [25] over names, descriptions and parameters. Claude Code's ToolSearch [2] searches by keyword weighted towards names, and GitHub Copilot [15] loads groups of tools on demand. Embedding search [10] and LLM routers [8] have been tried, for example in Goose [5]. In Hermes Agent 0.19.0, MCP tools are deferred once their definitions exceed 10% of the context window: `tool_search` returns compact hits, a name and one line each, `tool_describe` returns one full definition, and found tools are then called directly.

Decision models. A request to OpenRouter's Decisions API [23] carries a *state*, here the agent's search query, and one or more questions. A multiple-choice question lists up to 255 named options, each with a free-text description, and returns a probability per option; a yes/no question returns one probability. Microsoft Decision 1 is post-trained from Qwen3.5-9B [16]. TypeSafe Jev [28] matched it on our hand-written queries, putting the right tool first for 87.5% of them at 1,076 tools; we use Microsoft Decision 1 throughout.

3 The DM tool selector

Selector F answers each `tool_search` query in two rounds of DM calls (Figure 1).

Service step. One multiple-choice question, "Which service has the tool the agent needs?", lists a card for each of the 30 services and a "none" option. If $P(\text{none}) \geq 0.4$ the answer is empty. Otherwise the selector takes services in order of probability until they cover 90% of the probability, at most three, and always includes a service that the query names.

Tool step. For each chosen service, in parallel, two multiple-choice questions over its tools, each tool described by its name, the start of its description and its parameter names: "Which tool best serves what the agent wants to do?" and "Which tool would an expert engineer call first to do what the agent wants?". Scores are weighted by the service probability. The answer holds the five tools that best serve the query, with one-line descriptions, and then, as names only, up to three prerequisites from the second question and the chosen services' most-used tools, at most 15 tools in all. Tools already shown in the run are not repeated. The usage lists are built leave-one-task-out, so a task never sees usage from its own runs.

The shape follows how agents work: they move service by service and use three to four tools per service, most of them in later steps, so the answer lists the likely next tools by name, which costs few tokens, while describing only the best five. A search takes a median of 1.1 s.

4 Experimental setup

Tool catalog. We started each MCP server with dummy credentials and saved its `tools/list` response: names, descriptions and input schemas exactly as published. The catalog has 1,076 tools from 30 servers, including near neighbours such as GitLab beside GitHub, AWS CloudWatch beside Grafana and AWS EKS beside Kubernetes (App. B). No tool was ever executed against a real service.

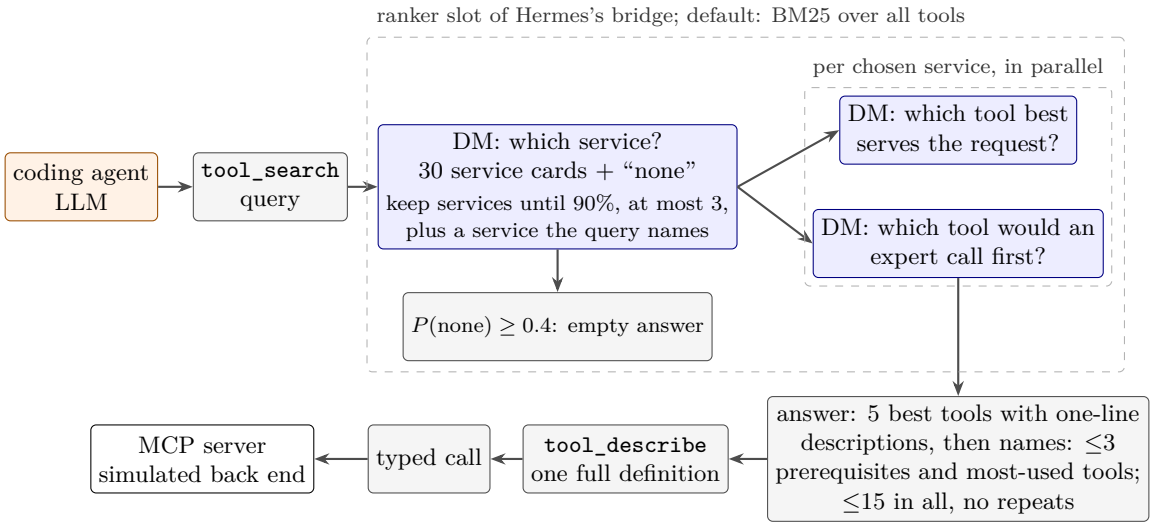


Figure 1: Selector F inside Hermes Agent's search bridge. The agent's query goes to the ranker slot. One DM call chooses the services or answers "none"; then two DM calls per chosen service run in parallel. The agent receives five compact hits and a list of names, reads full definitions with `tool_describe` and calls tools directly. In the baseline, BM25 fills the same slot; everything outside the slot is identical.

Tasks and simulated back ends. Each task is a small Python repository whose fix needs information that exists only in the services, such as a failing CI check, a Sentry stack trace or a decision in Slack. The 37 tasks do not name the service: 22 need one service, 7 need several, 4 ask for information that does not exist, where the right outcome is to stop and leave a `TODO`, and 4 need no catalog tool. Each task has rules that map a tool and matching arguments to a response; unmatched reads return nothing and unmatched writes are recorded for grading. A run passes when the hidden tests pass, every required write was made with the right arguments, and no destructive call was made.

Agents. We use Hermes Agent [20] 0.19.0 with its own search bridge over all 1,076 tools and replace only the ranker behind `tool_search`; the baseline is its built-in BM25. Agents run without credentials in isolated sandboxes (App. A). DeepSeek V4.1 Flash [7] is pinned to one provider with temperature 0.6 and GPT-6.1 Sol to OpenAI with default sampling. *Tokens* are all input, cached input and output tokens summed over a run's LLM calls.

5 Measuring token differences in agents

Pass rates are near ceiling for both rankers, so we compare tokens, and tokens are noisy: the standard deviation of the per-run log ratio between conditions was 0.55 with DeepSeek and 0.29 with Sol, so a single pair can swing by a factor of 1.7. Batches made at different times also drift: two BM25 batches a few days apart differed by 6%, with an interval from −13% to +2%. We therefore run both conditions on the same task and repeat back to back, a *pair*, take the log ratio of their tokens, average it per task, and test the 37 task means with a two-sided t-test, with a task-level bootstrap 95% interval. A verdict of "fewer tokens" needs $p < 0.05$ and an interval that excludes 1. Before each batch we wrote an analysis plan and ran the batch to completion regardless of interim numbers. Planned from the measured variance, DeepSeek ran 15 repeats per task, for 98% power to detect a 12.5% reduction, and Sol, which is less noisy, ran 5. For comparison at equal size we also

Table 1: Hermes Agent with its built-in BM25 and with selector F, 37 tasks, same-time pairs. Tokens, searches and LLM calls per run; tokens as mean / median. Change: selector F against BM25 in the same pairs, geometric mean of per-pair ratios averaged per task, with a task-level bootstrap 95% CI; p: t-test over the 37 task means. The last DeepSeek rows are a random five of its 15 repeats per task.

Agent model, repeats	Selector	Passed	Tokens	Change in tokens	p	Searches	LLM calls
DeepSeek V4.1 Flash, 15	BM25	550/555	791k / 553k			11.1	29.0
	selector F	550/555	680k / 507k	**−12.6%** [−18, −7]	0.0002	7.1	26.8
GPT-6.1 Sol, 5	BM25	172/185	193k / 168k			5.5	13.4
	selector F	173/185	171k / 133k	**−9.7%** [−16, −4]	0.004	2.2	11.5
DeepSeek V4.1 Flash, 5	BM25	184/185	769k / 552k			10.7	28.5
	selector F	182/185	717k / 477k	−8.5% [−16, +1]	0.07	7.3	27.4

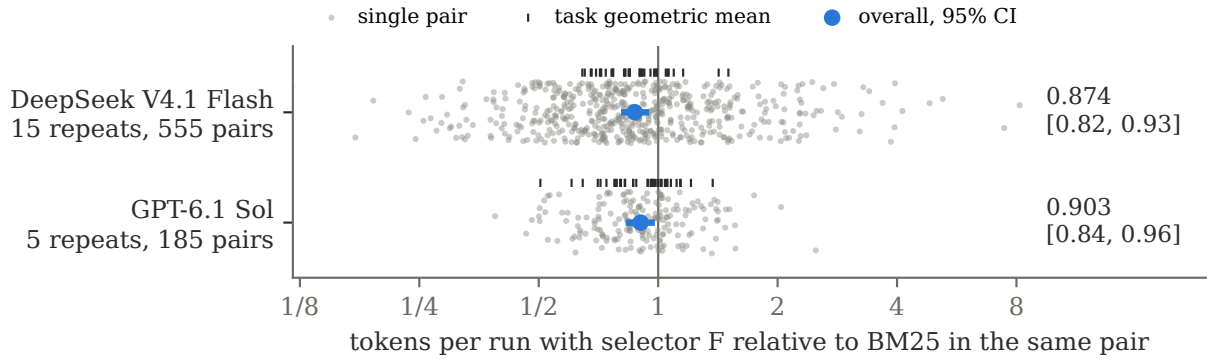


Figure 2: Tokens per run with selector F relative to BM25 in the same pair. Grey: single pairs; black ticks: the 37 task-level geometric means; blue: overall geometric mean of the task means with a task-level bootstrap 95% CI.

report five of DeepSeek's repeats per task, drawn at random with a seed fixed before the analysis.

6 Results

6.1 Tokens inside Hermes Agent

Selector F used fewer tokens than BM25 with both agent models (Table 1, Figure 2). With DeepSeek V4.1 Flash over 15 repeats the reduction was 12.6%, $p = 0.0002$; at five repeats, DeepSeek's higher noise leaves the interval just reaching 1. With GPT-6.1 Sol it was 9.7%, $p = 0.004$. The saving came from fewer turns: agents searched 36% less with DeepSeek and 60% less with Sol, and made 8% and 12% fewer LLM calls. Pass rates were equal with both models. The saving was largest on the 7 multi-service tasks, 24% with both models, against 12% with DeepSeek and 6% with Sol on the 22 single-service tasks.

6.2 Service errors

For 2,477 agent queries that name exactly one service, the DM's service step chose that service 97.9% of the time. Of its 51 errors, 47 were between Git, GitHub and GitLab, most often Git for a GitHub query, because the original service cards did not say that pull requests and branches live on GitHub

Table 2: Offline selector study at 1,076 tools with Microsoft Decision 1. Hand: hit@k on 120 hand-written queries. Agent: share of the next tools of 400 agent searches covered by the first 5 or all 15 tools, and share of searches with all next tools covered. Named: the named service ranked first, on 350 queries and on the 200 in the Git family. Variants 4–13 build on variant 3. Bold: best per column.

	Hand			Agent			Named	
Variant	hit@1	hit@5	hit@15	first 5	all 15	all covered	all	Git family
1 top service only	0.87	0.90	0.91	0.80	0.83	0.75	0.97	0.95
2 services until 90%	0.88	0.96	0.96	0.83	0.85	0.78	0.96	0.94
3 2 + fixed cards	**0.92**	0.98	0.99	0.83	0.86	0.79	0.98	0.97
4 "choose the named service"	**0.92**	0.98	0.99	0.83	0.86	0.79	0.98	0.97
5 "none" option	**0.92**	0.98	0.99	0.83	0.86	0.78	0.98	0.97
6 named-service rule	**0.92**	0.98	0.99	0.83	0.86	0.79	**1.00**	**1.00**
7 "expert first" question	0.83	**0.99**	**1.00**	0.83	0.85	0.78	0.98	0.98
8 yes/no per tool	0.75	0.97	**1.00**	0.82	0.86	0.80	0.98	0.97
9 sliding window, no service step	0.86	**0.99**	**1.00**	**0.84**	0.86	**0.81**	0.98	0.99
10 yes/no on all tools, no service step	0.63	0.86	0.95	0.73	0.83	0.76	0.67	0.67
11 "which tools are relevant"	0.88	0.98	0.99	0.83	0.86	0.79	0.98	0.97
12 best serves and expert first	**0.92**	0.98	**1.00**	0.83	0.86	0.79	0.98	0.97
13 best serves and relevant	**0.92**	0.98	0.99	0.83	0.86	0.79	0.98	0.97

rather than in the local Git copy. Selector F therefore uses fixed cards for these three services and always includes a service the query names, and it passed the GitHub pull-request task in all of its runs. The remaining failures were hard for both rankers: two monitoring tasks, `loki_errors` and `prom_threshold`, account for 8 of BM25's 13 failures with Sol and 10 of selector F's 12.

6.3 Designing the selector

We chose each element of selector F offline, comparing 13 variants at 1,076 tools on three query sets: 120 hand-written queries, plus 40 that no tool can serve; 400 searches made by agents in BM25 runs, labelled with the new tools the agent used after each search; and 350 agent queries that name exactly one service (Table 2).

What mattered. Taking services until 90% of the probability instead of only the top one raised hit@5 from 0.90 to 0.96, and fixing the GitHub, Git and GitLab cards raised hit@1 from 0.88 to 0.92. Always including a named service put it first on all 350 named queries. The "none" option answered "none" for 95% of the queries that no tool serves and never for one that a tool does.

What did not. Rephrasing the questions moved hit@1 by a few points at most. Asking yes or no per tool lowered hit@1 to 0.75 within services and to 0.63 over all tools: the DM compares options well but judges a single tool poorly. A sliding window over all tools, without a service step, covered slightly more next tools but needed about nine DM calls per search instead of two.

Held-out check. Selector F combines variant 3 with the "none" option, the named-service rule, the "expert first" question and usage lists. On 500 agent searches from a different batch it covered the most next tools, 0.845 within 15 against 0.806 for BM25 and 0.79 within the first five against 0.71 (Figure 3).

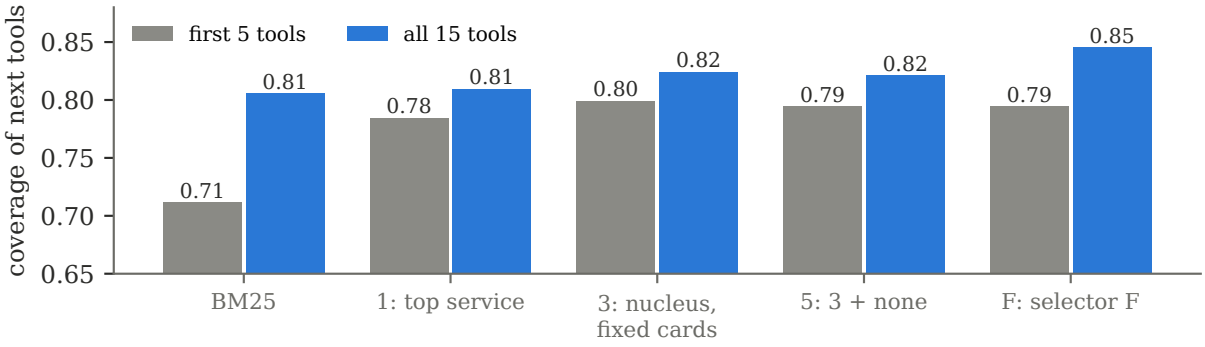


Figure 3: Held-out check on 500 agent searches: share of the next tools covered by the first 5 and by all 15 returned tools. Variant numbers as in Table 2. The vertical axis starts at 0.65.

Table 3: Offline retrieval on 120 hand-written queries. Hit@k: a right tool is among the first k.

	613 tools		1,076 tools	
Mechanism	hit@1	hit@5	hit@1	hit@5
Claude Code style	0.48	0.71	0.45	0.65
BM25	0.54	0.75	0.49	0.76
embeddings	0.51	0.80	0.47	0.75
LLM router	0.86	**0.98**	0.82	**0.98**
DM hierarchy	**0.93**	0.97	**0.88**	0.97

6.4 Against shipped mechanisms and an LLM router

On the 120 hand-written queries, a plain server-then-tool DM hierarchy ranked better than every shipped mechanism and than an LLM router that reads one line per tool (Table 3). Adding 463 tools from 18 more servers lowered hit@1 by 3–6 points for every mechanism and left the order unchanged.

6.5 External benchmark: ToolRet

To compare the DM with published tool retrievers, we ran the full ToolRet benchmark [26]: 7,961 queries over 44,453 tools from 35 datasets, scored by NDCG@10 averaged as in the paper. ToolRet's tools have no services, so the DM re-ranks candidates instead: the union of the top 50 tools from BM25 and from text-embedding-3-small. Method A asks one multiple-choice question over the 100 candidates with short tool cards; method E shows each tool's documentation in four parallel questions of 25 tools and a final question over the six best of each. Both ask "A retrieval system is ranking tools for a user's task. Which tool is most relevant to it?". ToolRet has two settings: with an instruction, a sentence written for each query by GPT-4o with the target tools in view, and with the query alone. Our BM25 reproduced the paper's: 36.4 against 36.5 with the instruction and 22.2 against 22.3 without.

The DM was best in both settings (Table 4). With the instruction, method E scored 53.6, 2.3 points above the best published system, a retriever and re-ranker trained for tool retrieval. With the query alone, the setting closest to an agent's search, method A scored 41.0 against 35.5 for the best published re-ranker. The methods were fixed on a 385-query pilot; on the remaining queries the scores were the same, 53.8 and 41.6.

Table 4: ToolRet, all 7,961 queries: NDCG@10 averaged over the three categories as in ToolRet. Published rows are quoted from ToolRet [26] and, for Tool-Embed, Tool-Rank and Qwen3, from Tool-REX [14], which reports only the setting with instruction. The DM rows re-rank the union of BM25 and embedding candidates; the published re-rankers re-rank the top 100 of NV-Embed-v1 or Tool-Embed-4B.

System	with instruction	without instruction
DM, method E	**53.6**	–
DM, method A	51.5	**41.0**
Tool-Embed-4B + Tool-Rank-4B, trained on ToolRet's documents	51.2	–
Tool-Embed-4B, trained	49.2	–
bge-reranker-v2-gemma	47.5	35.5
jina-reranker-v2-base	47.2	33.6
Qwen3-Embedding-8B	46.2	–
NV-Embed-v1	42.7	33.8
gpt-3.5-turbo-1106 as re-ranker	38.8	30.8
text-embedding-3-small, our first stage	38.1	24.7
BM25	36.4	22.2

7 Discussion

Fewer searches matter more than better hits alone. A search costs the agent a full turn: the model re-reads its context to write the query and again to read the answer. Selector F's answers are longer than BM25's, yet runs used fewer tokens, because a better first answer, with the right service's likely next tools listed by name, saves the turns that rephrased queries would take.

What "universal" means here. Selector F needs only the query and the catalog, no training, and slots into any harness that has a search tool; the same DM, asked the same kind of question, also re-ranked a 44k-tool catalog without services. We have not shown that it helps in every harness, catalog or kind of task.

What the DM is good and bad at. The DM is good at comparing options, a service among cards or a tool among a service's tools, and poor at judging one tool in isolation. Its answers depend on the option texts: the Git-family errors came from service cards, not from the model.

Threats to validity. The services are simulated; real services return longer, messier results that could change how agents search. The tasks are coding tasks, two agent models were tested, and the agent runs used one harness. Selector F was designed on separate data before the agent runs and checked on held-out searches; its usage lists come from the same task suite, built leave-one-task-out. The baseline is Hermes Agent's BM25 as shipped; a tuned BM25 or query rewriting might narrow the gap. One task, `k8s_crashloop`, was excluded for both conditions before the runs because its grading rule forbids a reasonable read-only command; no run was dropped for its result.

8 Related work

Retrieving tools before generation is the standard answer to large catalogs [6, 24]. AnyTool [8] selects among over 16,000 APIs with a hierarchy of LLM agents, category first and tool second, and MCP-Zero [9] brings the server-then-tool idea to MCP; selector F applies the same hierarchy with a decision model. RAG-MCP [10] and Mudunuri et al. [19] retrieve MCP tools with embeddings.

ToolRet [26] finds that general-purpose retrievers do poorly on tools, Tool-REX [14] trains a retriever and re-ranker on LLM-expanded tool documents, and Lee [12] find that selecting a tool is largely retrieval. MCP benchmarks such as LiveMCPBench [17], MCPMark [29], MCP-Atlas [4] and Toolathlon [13] score agents' task completion with real tools; we fix the agent and vary only the selector. Vendor reports compare tool search on thousands of tools by selection accuracy [11, 27], and the Artificial Analysis Decision Index [3, 21] compares DMs with LLMs on standalone decisions; we place a DM inside agents and measure whole runs.

9 Conclusion

A decision model can serve as an agent's tool selector. Built on Microsoft Decision 1, selector F chooses services with a "none" option and then tools, in about three DM calls per search. Inside Hermes Agent, in pre-planned same-time paired runs, it used 12.6% fewer tokens than BM25 with DeepSeek V4.1 Flash and 9.7% fewer with GPT-6.1 Sol, with 36–60% fewer searches and equal pass rates. Offline it put the right tool first far more often than BM25, and on ToolRet the same model re-ranked candidates better than every published retriever and re-ranker. Testing other harnesses, real services and non-coding work is the next step.

Reproducibility. We will release the tool harvester, catalog snapshots, simulator, tasks with hidden tests and reference solutions, the selector and ranking server, the Hermes patch, the sandbox setup, all run records, the analysis plans and the analysis and figure scripts.

References

[1] Anthropic. Tool search tool (Claude platform documentation; regex and BM25 variants). https://platform.claude.com/docs/en/agents-and-tools/tool-use/tool-search-tool, 2025. Accessed 6 November 2026.

[2] Anthropic. Claude code documentation: Scale with MCP tool search. https://code.claude.com/docs/en/mcp, 2026. Accessed 6 November 2026.

[3] Artificial Analysis. Artificial analysis decision index. https://artificialanalysis.ai/evaluations/artificial-analysis-decision-index, 2026. Accessed 6 November 2026.

[4] Chaithanya Bandi et al. MCP-Atlas: A large-scale benchmark for tool-use competency with real MCP servers. *arXiv preprint arXiv:2602.00933*, 2026.

[5] Block. goose: Tool selection strategy (vector and LLM-based tool routers, preview feature). https://web.archive.org/web/20251031035432/https://block.github.io/goose/docs/guides/managing-tools/tool-router/, 2025. Preview in 2025. The vector strategy was removed in https://github.com/block/goose/pull/3933 (August 2025), the LLM-based strategy and its documentation in https://github.com/block/goose/pull/6250 (December 2025).

[6] Huamin Chen, Xunzhuo Liu, Junchen Jiang, Bowei He, and Xue Liu. Outcome-aware tool selection for semantic routers: Latency-constrained learning without LLM inference. *arXiv preprint arXiv:2603.13426*, 2026.

[7] DeepSeek. DeepSeek V4.1 Flash. https://openrouter.ai/deepseek/deepseek-v4.1-flash-1017, 2026. OpenRouter model page; model identifier deepseek/deepseek-v4.1-flash-1017.

[8] Yu Du, Fangyun Wei, and Hongyang Zhang. AnyTool: Self-reflective, hierarchical agents for large-scale API calls. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *PMLR*, pages 11812–11829, 2024. arXiv:2402.04253.

[9] Xiang Fei, Xiawu Zheng, and Hao Feng. MCP-Zero: Active tool discovery for autonomous LLM agents. *arXiv preprint arXiv:2506.01056*, 2025.

[10] Tiantian Gan and Qiyao Sun. RAG-MCP: Mitigating prompt bloat in LLM tool selection via retrieval-augmented generation. *arXiv preprint arXiv:2505.03275*, 2025.

[11] Eric Gustin. We threw 4,000 tools at Anthropic's new tool search. Here's what happened. https://www.arcade.dev/blog/anthropic-tool-search-4000-tools-test/, December 2025. Arcade.dev blog, 3 December 2025.

[12] Janghoon Lee. Selection is retrieval, abstention is not: On-device tool routing over 70 Korean-English actions. *arXiv preprint arXiv:2609.18672*, 2026.

[13] Junlong Li et al. The tool decathlon: Benchmarking language agents for diverse, realistic, and long-horizon task execution. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.25726.

[14] Xuan Lu, Haohang Huang, Rui Meng, Yaohui Jin, Wenjun Zeng, and Xiaoyu Shen. Tools are under-documented: Simple document expansion boosts tool retrieval. In *International Conference on Learning Representations (ICLR)*, 2026. arXiv:2510.22670.

[15] Microsoft. Use tools with agents (Visual Studio Code documentation: 128-tool limit and virtual tools). https://code.visualstudio.com/docs/copilot/agents/agent-tools, 2026. Accessed 6 November 2026.

[16] Microsoft. Not every decision needs an LLM: Getting started with Microsoft-Decision-1. https://techcommunity.microsoft.com/blog/educatordeveloperblog/not-every-decision-needs-an-llm-getting-started-with-microsoft-decision-1/4563468, October 2026. Microsoft Community Hub, Educator Developer Blog.

[17] Guozhao Mo, Wenliang Zhong, Jiawei Chen, Qianhao Yuan, Xuanang Chen, Yaojie Lu, et al. LiveMCPBench: Can agents navigate an ocean of MCP tools? *arXiv preprint arXiv:2508.01780*, 2025.

[18] Model Context Protocol. Model context protocol specification. https://modelcontextprotocol.io/specification/2025-06-18, 2025.

[19] Sarat Mudunuri, Jian Wan, Ally Qin, and Srinivasan Manoharan. Semantic tool discovery for large language models: A vector-based approach to MCP tool selection. *arXiv preprint arXiv:2603.20313*, 2026.

[20] Nous Research. Hermes agent. https://github.com/NousResearch/hermes-agent, version 0.19.0 (PyPI), 2026.

[21] Randal S. Olson. When to use a decision model instead of an LLM. Blog post, 15 October 2026, https://www.randalolson.com/2026/10/14/when-to-use-a-decision-model-instead-of-an-llm/, 2026.

[22] OpenAI. Codex CLI. https://github.com/openai/codex, 2026. MCP tool search with BM25 in codex-rs/core/src/tools/handlers/tool_search.rs; source checked 6 November 2026.

[23] OpenRouter. Decisions API reference. https://openrouter.ai/docs/api/api-reference/alphadecisions/submit-a-decisions-request, 2026. Accessed 6 November 2026.

[24] Yujia Qin et al. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2307.16789.

[25] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[26] Zhengliang Shi, Yuhan Wang, Lingyong Yan, Pengjie Ren, Shuaiqiang Wang, Dawei Yin, and Zhaochun Ren. Retrieval models aren't tool-savvy: Benchmarking tool retrieval for large language models. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 24497–24524, 2025. arXiv:2503.01763.

[27] Stacklok. Stacklok's MCP optimizer vs Anthropic's tool search tool: A head-to-head comparison. https://stacklok.com/blog/stackloks-mcp-optimizer-vs-anthropics-tool-search-tool-a-head-to-head-comparison/, 2025.

[28] TypeSafe AI. Jev 1.14 decision model. https://typesafe.ai; served via OpenRouter as typesafe/jev-1.14, https://openrouter.ai/typesafe/jev-1.14, 2026.

[29] Zijian Wu, Xiangyan Liu, Xinyuan Zhang, Lingjun Chen, Fanqing Meng, Lingxiao Du, et al. MCPMark: A benchmark for stress-testing realistic and comprehensive MCP use. *arXiv preprint arXiv:2509.24002*, 2025.

A Sandbox hardening

The isolation of §4 consists of the following measures.

- Agents run as an unprivileged user with an empty environment and no credentials; experiment files are readable only by root and the API key is held by a local proxy that admits only the experiment models, because harness-level permissions alone let an agent read hidden tests through absolute paths.
- Each Hermes run gets its own mount namespace with a private work directory, home directory and `/tmp`, so runs cannot see each other's files or leftovers.
- The MCP server accepts only requests that carry a per-run token the agent's shell cannot read, so the shell cannot list the catalog or query the ranker directly.
- Hermes's deny list blocks probing commands: `curl`, `wget`, `ps`, `ss`, `netstat`, `nmap`, `lsof`, `/proc`, `localhost` and `find /`.
- Rankers are compared inside Hermes's own bridge rather than through a separate search tool with a generic call tool, so that only the ranking differs.
- The provider and sampling are pinned as described in §4.

B Tool catalog and task set

Table 5: The real tool catalog: tools per MCP server, from each server's own `tools/list`.

Catalog	Servers with their numbers of tools	Tools
main	Linear 198, Atlassian 98, GitHub 91, Grafana 81, Git 28, MongoDB 27, Playwright 25, Notion 24, Kubernetes 23, Sentry 9, Slack 8, Postgres 1	613
extra	GitLab 127, monday 84, Salesforce 74, Heroku 33, HubSpot 21, AWS CloudWatch 19, AWS EKS 16, Airtable 16, Filesystem 14, CircleCI 13, Browserbase 9, Memory 9, Netlify 9, Terraform 9, Elasticsearch 4, ClickHouse 3, Figma 2, Docker 1	+463
all	30 servers	1,076

Table 6: Task set v2, 38 tasks. Prompts are shortened to their first sentence and shown without the escape clause. Services: the servers of the needed tools, in order; $+w$: number of required writes; $\star$: new in v2.

Category	Task	First sentence of prompt	Services
control	`c_chunk`	chunk in batching/chunk.py drops the last partial chunk.	–
control	`c_dedupe`	dedupe in collections_util/dedupe.py loses the original order.	–
control	`c_duration`	parse_duration in timeutil/duration.py only understands hours.	–
control	`c_leap`	is_leap in calendar_util/leap.py is wrong for some years.	–
single	`confluence_limits`	Our public API rate limits changed.	atlassian
single	`eks_importer` $\star$	The data team's nightly batch-importer job has failed three nights in a row.	aws_eks
single	`gh_ci_fail`	Pull request #214 in acme/payments can't be merged because a check is red.	github $+1w$
single	`gh_codesearch`	SUPPORTED_CURRENCIES in shop/currencies.py has drifted from the company-wide canonical list, which lives in another repository.	github
single	`gh_dependabot`	Security says acme/payments has dependencies with known vulnerabilities.	github
single	`gh_issue`	Please implement acme/payments#301.	github
single	`gh_open_pr`	Pagination in acme/catalog is broken (see #57).	github $+2w$
single	`gh_release`	We are behind on the ledger API.	github
single	`gh_review`	There is reviewer feedback waiting on pull request #88 in acme/ledger.	github $+1w$
single	`grafana_incident`	We are in the middle of an incident about checkout timeouts and the responders agreed on a mitigation.	grafana $+1w$
single	`jira_impl`	Pick up PAY-412: implement it, then move it to review.	atlassian $+1w$
single	`k8s_crashloop`	ledger-worker keeps restarting in production (namespace ledger) since the last config change.	kubernetes
single	`k8s_oom`	report-builder (namespace reports) keeps dying in production.	kubernetes

Category	Task	First sentence of prompt	Services
single	`linear_bug`	Fix ENG-77 and move it to In Review.	linear +1w
single	`loki_errors`	Customers stopped getting shipping e-mails at around 06:00 UTC today; the notifier service is the suspect.	grafana
single	`mongo_shape`	The nightly export has started crashing on recent orders.	mongodb
single	`notion_runbook`	We are rotating the webhook signing keys.	notion
single	`oncall_contact`	Set ESCALATION_CONTACT in ops/escalation.py to the e-mail address of whoever is on call for the payments team right now.	grafana
single	`pg_migration`	Add migrations/0042_payments_merchant_idx.sql with an index named payments_merchant_created_idx for looking up a merchant's payments by creation time.	postgres
single	`prom_threshold`	Set SLOW_REQUEST_MS in alerts/config.py to twice the checkout service's current p99 request latency, in milliseconds.	grafana
single	`sentry_events`	Since this morning's deploy some clients get HTTP 500 from POST /refunds.	sentry
single	`sentry_fix`	The noisiest production error of payments-api right now needs to go.	sentry +1w
single	`slack_decision`	The payments team agreed on a new webhook retry policy in a chat discussion.	slack +1w
multi	`cw_reports` ⋆	The legacy reports API has been answering 502 since yesterday evening.	aws_cloudwatch, slack +1w
multi	`gl_pipeline` ⋆	The main-branch pipeline of billing-legacy is red.	gitlab +1w
multi	`linear_notion_pr` ⋆	Do ENG-91 the way our API guidelines say, and open a pull request for it from a branch named eng-91 against main in acme/public-api.	linear, notion, github +1w
multi	`multi_alert`	Something is paging for the checkout service.	grafana, slack +1w
multi	`multi_jira_conf`	Implement PAY-530 as its design document specifies, then leave a one-line summary on the ticket.	atlassian +1w
multi	`multi_sentry_gh`	A recent change to ledger-worker introduced a reconciliation bug that shows up as a production error.	sentry, github +1w
multi	`promo_sentry_jira` ⋆	Android users report they can't apply promo codes.	sentry, atlassian +1w
absent	`absent_jira`	Implement PAY-999 in payments/limits.py.	–
absent	`absent_release`	Pin SDK_VERSION in sdk/version.py to the newest release of acme/ghost-sdk (without the leading v).	–
absent	`absent_sentry`	The billing-cron error someone mentioned in standup needs fixing in billing/cron.py.	–
absent	`absent_slack`	Set CACHE_TTL_S in cache/config.py to the value the platform team agreed on.	–